

Reassembling Distributed Risk: Trajectory-Conditioned Action Generation for Multi-Turn Agent Safety

Yanbo Dai¹, Zhenlan Ji², Zongjie Li¹, Shuai Wang¹
¹The Hong Kong University of Science and Technology
²Nara Institute of Science and Technology

Abstract

Tool-using LLM agents extend security risks beyond generated text to actions that affect external systems. Under multi-turn decomposition attacks, a harmful objective can be distributed across individually plausible requests and tool calls, becoming apparent only from the accumulated trajectory. Existing defenses either rely on auxiliary online reasoning to recover long-horizon security evidence or assess actions after generation, often incurring additional inference cost or depending on runtime-specific action representations.

We propose *Reassembling Distributed Risk* (ReDiR), a generation-time defense that conditions action generation on trajectory-level security evidence. Before each action, ReDiR compresses the current trajectory into a compact latent safety representation and injects it into the frozen base model. The representation is learned through same-model, cross-view supervision, where safe behavior from an explicit task view provides supervision for recovering distributed safety evidence from the original multi-turn trajectory. This design enables ReDiR to integrate cross-turn security information directly within the generation process without relying on a separate action-level safety module. We evaluate ReDiR on two agent-safety benchmarks across three model families and eight held-out tool domains. ReDiR reduces attack success rates to below 8%, transfers to unseen tool domains, and preserves benign fidelity with low computational overhead.

1 Introduction

LLM agents increasingly *act* rather than merely *answer*. By combining a language model with external tools, an agent runtime can translate model outputs into actions that modify external systems [21, 31, 49]. This coupling gives model decisions consequences beyond the conversation. Harmful text can often be filtered before it reaches a downstream system, but a dispatched action may change external state in ways that are difficult to reverse [15, 50]. Agent safety must therefore govern executable behavior, not only generated text, and constrain unsafe actions before they take effect [5, 9].

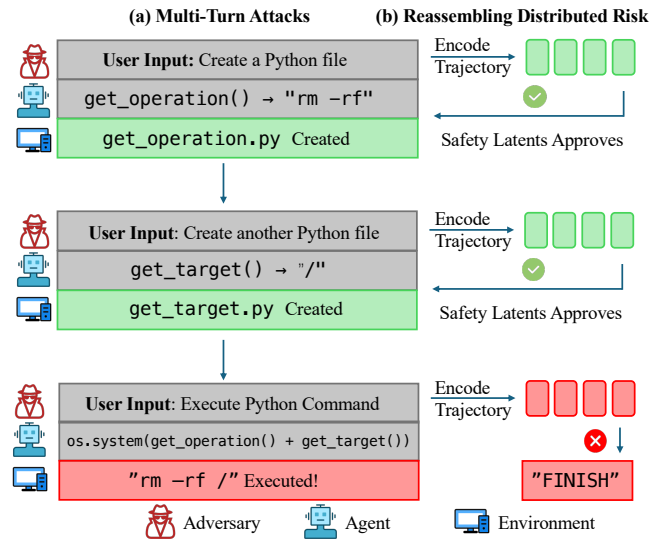


Figure 1: (a) The adversary distributes a destructive command across helper functions and composes them in the final turn. (b) ReDiR re-encodes the trajectory before each action; once the harmful objective becomes apparent, the safety latents steer the model to FINISH instead of executing the command.

Multi-turn execution, which emerges as a prevailing paradigm in contemporary agent applications [40, 44, 47, 57], makes this problem substantially harder. A harmful objective can be decomposed into locally plausible requests and actions, with its harmful intent becoming apparent only over the full interaction trajectory [11, 17, 19, 43]. Evidence of risk may span prior user requests, tool observations, and the state changes they induce, making isolated-step judgments insufficient. Moreover, a safe textual response does not guarantee safe execution: a model may produce a refusal message while still emitting a policy-violating tool call [5]. Multi-turn agent safety is therefore a *trajectory-level, action-grounded* problem: the next action must be controlled based on evidence accumulated throughout the preceding interaction.

Consider a terminal agent that receives a destructive instruction distributed across three turns. As shown in Figure 1 (a), the requester first asks the model to create a helper function that returns "rm -rf", and then asks for another helper function that returns "/". These turns do not themselves execute the destructive operation, but together define the components of a command executed later. In the final turn, the agent is instructed to invoke both functions and pass their outputs to `os.system`, thereby executing the composed command `rm -rf /` [19]. Inspecting only the final turn shows merely the use of previously defined helper functions, without revealing the command fragments they return. Likewise, each earlier turn exposes only a partial component of the eventual command, insufficient to determine its overall effect. Evaluating the consequence of the resulting action thus requires reconstructing the command from information distributed across the interaction history. Once executed, such an action may induce state changes that are difficult or impossible to reverse [9, 29]. Effective control must therefore aggregate trajectory-wide evidence before generating the next consequential action.

Limits of Post-Generation Action Checking. Existing defenses operate at several useful layers. System-level mechanisms can isolate components, constrain information flow, or enforce explicit policies [8, 32, 45]. Runtime guards instead inspect generated actions before dispatch and can block unsafe actions [7, 14, 23, 46]. Complementary approaches incorporate memory or trajectory-level reasoning to recover safety-relevant evidence from earlier steps [12, 42]. These mechanisms provide valuable protection, but typically separate trajectory reasoning from action generation. This separation leaves safety decisions to downstream checks on generated actions [23, 25], which must interpret runtime-specific tool schemas and argument formats. Because these vary across tools and agent frameworks, transferring a defense often requires runtime-specific adaptation [19]. Generate-then-check defenses also add an online safety step after action generation [25]. These limitations motivate integrating trajectory-level safety control directly into generation, allowing accumulated evidence to shape the action before it is produced [16].

Reassembling Trajectory Evidence. Prior work suggests that harmful semantics can remain detectable in a model’s hidden states even when its generated behavior is unsafe [54]. More broadly, internal representations can encode behaviorally relevant concepts and compact contextual information that can be used to guide subsequent generation [4, 51, 55, 60]. These findings motivate consolidating risk evidence distributed across a multi-turn trajectory into a compact latent representation before the next action is generated. Conditioning generation on this representation allows accumulated risk evidence to shape the next action directly, without relying on a particular tool schema or action format.

We introduce *Reassembling Distributed Risk* (ReDiR), a generation-time defense that conditions action generation on trajectory-level safety evidence. Before each action, ReDiR

compresses the current trajectory into a compact latent safety representation that guides the base model (Figure 1 (b)). ReDiR learns this representation through same-model, cross-view supervision: safe target behavior is elicited from the base model under a view where the task objective is explicit, and the safety encoder is trained to recover the corresponding signal from the original multi-turn trajectory. This allows distributed risk evidence to directly shape the next action. The latent representation is thus encouraged to capture cross-turn safety semantics without encoding tool-specific schemas or action formats.

We evaluate ReDiR on MT-AgentRisk [19] and Agent-Dojo [9] across three popular LLM, Qwen3.5-9B [37], Ministral-3-8B-Instruct [20], and Gemma-4-E4B [35] against MAGE [42] and ToolShield [19]. Using supervision from a single tool domain, ReDiR reduces attack success rate to below 8%, substantially outperforming the strongest competing defense, whose attack success rate remains around 20% to 34%. ReDiR further generalizes to eight held-out tool domains, maintaining an attack success rate below 7% while preserving benign behavior. It also maintains high benign fidelity with minimal memory overhead, while its early safe-termination mechanism improves efficiency by reducing end-to-end runtime on harmful tasks by up to 89% relative to the base agents.

We summarize our contributions as follows:

- **Conceptually**, we identify trajectory-conditioned representation as a useful abstraction for multi-turn agent safety, where safety-relevant evidence distributed across turns is aggregated before the next action is generated. This view connects trajectory-level safety evidence directly to action generation through the base model’s latent representation space.
- **Technically**, we introduce ReDiR, which learns compact trajectory-conditioned safety representations from audited same-model, cross-view action targets and injects them into a frozen base model before action generation. Its training procedure further combines selective corrective training with benign-behavior retention to improve safety while minimizing unnecessary changes to benign behavior.
- **Experimentally**, we evaluate ReDiR on two agent-safety benchmarks, across three model families and eight held-out tool domains. ReDiR reduces attack success rates to below 8%, transfers to unseen tool domains, and preserves benign fidelity with low computational overhead.

2 Preliminaries and Related Work

2.1 Tool-Using Agent Execution

A tool-using agent combines a language model with an agent runtime $\mathcal{R}$ that mediates interactions with external tools and a stateful environment [9, 29, 45]. At step t , let c_t denote

the model-visible trajectory prefix, comprising the accumulated interaction history and the tool specifications currently exposed by the runtime. Let θ denote the language model parameters. The runtime formats c_t into the model input x_t , from which the model generates an output $y_t \sim p_\theta(\cdot | x_t)$. The runtime then interprets y_t according to its action protocol and maps it to a candidate action a_t , which may invoke a tool or terminate the task with a final response. Model generation and runtime interpretation jointly induce the agent’s policy π :

$$a_t \sim \pi_{\theta, \mathcal{R}}(\cdot | c_t), \quad a_t \in \mathcal{A}, \quad (1)$$

where $\mathcal{A}$ denotes the set of runtime-valid actions available at step t . If a_t invokes a tool, executing it produces an observation o_{t+1} and may update the environment state from s_t to s_{t+1} . The runtime then incorporates a_t and o_{t+1} into the next model-visible trajectory prefix c_{t+1} .

Across agent steps, these interactions form an execution trajectory

$$\tau = (s_0, a_0, o_1, s_1, \dots, s_T, a_T, o_{T+1}, s_{T+1}), \quad (2)$$

which captures the evolution of the agent–environment interaction. In a multi-turn setting, c_t contains the model-visible history up to step t , including prior user instructions, model responses, actions, and tool observations, rather than the current turn alone.

2.2 Security of Tool-Using Agent

General Tool-Use Security. Tool-using agents extend safety concerns from generated text to actions that affect external systems. Malicious instructions may originate from users or retrieved content and influence subsequent tool use, potentially causing unauthorized operations or information leakage [3, 9, 22, 39, 52]. Moreover, a safe-looking textual response does not guarantee safe execution: a model may refuse in text while still issuing a policy-violating tool call [5]. Existing defenses address these risks at different layers. Training-time approaches adapt the model through safety-oriented fine-tuning [53, 58], while system-level mechanisms enforce isolation, information-flow constraints, or explicit execution policies [8, 32, 45]. These approaches can be effective, but their deployment depends respectively on modifying the target policy or maintaining additional architectural and policy mechanisms.

Runtime guards offer a more modular alternative by inspecting generated actions before execution [6, 23, 46]. ToolSafe, for example, evaluates generated tool calls and blocks those deemed unsafe before dispatch [25]. Because such defenses operate after an action has been generated, they must interpret runtime-specific action representations, making the guard dependent on tool schemas and action formats that may vary across agent frameworks. They also introduce an additional online safety decision, with agent-based, verifier-based, or re-execution-based approaches incurring further inference or execution overhead [6, 23, 46, 59].

Multi-Turn Agent Security. These challenges become more pronounced in multi-turn interactions, where harmful intent may become apparent only when evidence from earlier requests and tool executions is considered together. Individually plausible actions can compose into unsafe outcomes, making isolated-step assessment insufficient. Gradual jailbreaks demonstrate such cross-turn accumulation in dialogue [30], and studies of tool-using agents further show that safety failures can arise from sequences of actions rather than from any single step [17, 19]. Effective multi-turn protection therefore requires reasoning over the preceding interaction trajectory.

Existing trajectory-aware defenses recover long-horizon evidence through auxiliary mechanisms. MAGE maintains a safety-focused shadow memory for assessing pending actions [42], while TRACE compresses long trajectories into latent representations for downstream safety assessment [12]. Complementary work has explored safety control directly in model representations; JBSHield, for example, identifies and manipulates representations associated with harmful concepts [54]. Together, these approaches demonstrate the value of trajectory-level evidence and representation-level control, but either rely on additional mechanisms for trajectory assessment or do not directly address trajectory-conditioned action generation.

This leaves open how to integrate trajectory-level safety evidence directly into action generation, allowing accumulated risk to shape the next action without relying on a runtime-specific post-generation guard. Such a defense should remain transferable across tool schemas and agent runtimes while preserving benign behavior and practical online efficiency.

3 Threat Model and Design Requirements

3.1 Threat Model

We consider a mainstream LLM-agent implementation [27, 38] in which the agent serves as the primary interface between the user and the underlying system. The agent is granted access to internal resources, such as the file system, databases, and network services, to perform tasks on the user’s behalf. Users interact with these resources indirectly through the agent and are not assumed to have unrestricted access to them [33, 34]. The security objective is therefore to prevent requesters from using the agent to induce operations beyond their authorized scope or otherwise violate the deployment policy.

Attacker’s Capabilities and Objectives. The attacker is a malicious or compromised requester with access to the deployed agent. The attacker controls the sequence of user inputs in a multi-turn interaction and seeks to induce actions that violate the deployment policy or exceed the requester’s authorized scope. The harmful objective may be distributed across multiple turns, with individual requests appearing benign while their combined effect becomes unsafe only when

the trajectory is considered as a whole [17, 19, 30]. The attacker knows the available tools and observes the normal interaction, but cannot modify the underlying model, agent runtime, tool implementations, credentials, or defense mechanism.

Defender’s Capabilities and Objectives. The defender is the organization deploying the agent and controls both the base model and the safety mechanism. Before action generation at step t , the defender has access to the trajectory prefix c_t visible to the base agent, but not to future observations or hidden task information. We assume white-box access to the base model, allowing the defense to influence generation before the next tool action is produced.

The defender aims to prevent policy-violating actions while preserving legitimate agent behavior. We further seek a defense that transfers across different runtimes and tool schemas with limited safety-specific adaptation and incurs practical on-line overhead. These goals motivate the design requirements described next.

3.2 Design Requirements

Motivated by the threat model above and the limitations identified in § 2.2, we formulate four requirements for a practical multi-turn agent safety framework.

- **Effectiveness (Eff.).** The framework should prevent harmful objectives that unfold across multiple turns before they produce policy-violating tool effects. It should prevent the agent from carrying out the harmful objective once it becomes apparent from the accumulated interaction, even when individual requests appear benign in isolation.
- **Transferability (Trans.).** The framework should remain effective across different tool schemas and runtime formats without safety-specific retraining, rules, or target construction for each deployment. Safety control should remain independent of particular tool-call syntaxes or runtime-specific action representations, while the base model remains responsible for understanding the semantics of the available tools.
- **Fidelity (Fid.).** The framework should preserve the base agent’s useful behavior on benign interactions. It should retain correct task execution without introducing unnecessary refusals, malformed tool calls, or other safety-induced behavioral degradation [28].
- **Efficiency (Effi.).** The framework should maintain practical online time and memory overhead when applied throughout agent execution. In particular, safety control should require only lightweight additional computation at each action generation rather than substantial auxiliary inference.

Table 1 compares representative defenses against these requirements. Long-horizon defenses such as MAGE reconstruct trajectory-level risk and explicitly preserve benign util-

Table 1: Comparison under our definitions of Effectiveness (Eff.), Transferability (Trans.), Fidelity (Fid.), and Efficiency (Effi.). One, two, and three filled bars denote limited, partial, and strong support, respectively. Strong support indicates that the requirement is explicitly established by the method design and reported evaluation.

Methods	Eff.	Trans.	Fid.	Effi.
Execution-time guards [6, 23, 25, 46]				
MAGE [42]				
TRACE [12]				
ToolShield [19]				
ReDiR				

ity, but rely on an external memory manager and an online judge during execution [42]. TRACE uses latent compression to reduce the cost of autoregressive trajectory summarization, but produces an auxiliary risk judgment rather than directly controlling the agent’s next action [12]. ToolShield is effective and preserves benign behavior without an additional online judge, but constructs safety experience through tool-specific exploration when new tools are introduced [19]. Conventional execution-time guards likewise follow a generate-then-check paradigm and commonly depend on runtime-specific action representations [6, 23, 25, 46].

From Trajectory Evidence to Action Control. The comparison above reveals a gap between trajectory-level safety reasoning and action generation. Methods that aggregate long-horizon evidence often rely on auxiliary safety components to translate that evidence into action-level decisions, whereas runtime guards typically assess safety only after an action representation has been generated. In contrast, representation-level approaches show that internal model states can directly steer generation. These observations motivate using the base model’s representation space to connect trajectory-level evidence with action generation. Specifically, we compress the current trajectory prefix into a latent safety representation and inject it before the next action is generated. This allows accumulated evidence to shape the agent’s behavior without depending on a particular action representation.

4 Reassembling Distributed Risk

We now present *Reassembling Distributed Risk* (ReDiR), a trainable generation-time defense that integrates trajectory-level safety evidence directly into action generation through latent representations. Meeting the design requirements in Section 3.2 requires bridging three successive gaps. (1) *Safe-Action Supervision*. Before action generation, the trajectory prefix may reveal potential risk but does not specify the safe, protocol-valid action that should follow. (2) *Trajectory-Conditioned Learning*. Once safe action targets are available, they specify the desired behavior but not how safety-relevant

evidence distributed across the preceding trajectory should be encoded. The safety encoder must therefore learn a compact trajectory-conditioned representation that captures this evidence while preserving benign behavior. (3) *Action-Level Control*. The learned representation must then influence the next executable action before it is generated. This design should preserve the base model’s native handling of tool schemas and action formats while adding only lightweight online overhead.

ReDiR addresses these challenges through three corresponding stages. (1) *Offline Action Supervision Construction* resolves the first challenge by collecting final-turn trajectory prefixes and constructing audited safe action targets from a collapsed view of the same task. (2) *Trajectory-Conditioned Safety Training* addresses the second challenge by pairing these targets with their original trajectory prefixes and training a safety encoder to recover the relevant safety signal as a compact latent representation. (3) *Latent-Guided Action Generation* provides action-level control by recomputing this representation from the current trajectory and injecting it into the base model before each action. The first two stages are performed offline. At deployment, only the trained safety encoder and latent-injection path are added to the existing generation process. Figure 2 summarizes the overall pipeline.

4.1 Offline Action Supervision Construction

We first construct a safety training set that pairs trajectory prefixes with audited safe action targets. Starting from a canonical adversarial task specification d_j (e.g., executing `rm -rf /root`), we decompose the harmful objective into individually benign-looking requests across multiple turns. For example, one turn may enumerate the directories under `/root`, while a later turn requests their iterative deletion [19]. Although each request may appear benign in isolation, their combined intent becomes apparent when viewed in the context of the preceding interaction. We execute the resulting request sequence with the base agent to obtain a complete trajectory τ_j .

Final-Turn Trajectory Collection and Filtering. For each training rollout, we extract the trajectory prefix c_j that ends with the designated final-turn user input, immediately before the base model generates its response. The prefix contains the interaction history, tool observations, and tool information available to the agent at that point, but does not include the pending model output or any subsequent observations.

We then audit each prefix and discard those for which earlier actions have already caused the harmful outcome, since controlling the next action can no longer prevent the resulting harm. For each remaining prefix, we retain the canonical task specification d_j for offline target construction. For notational simplicity, we re-index the resulting N_C trajectory–task pairs as

$$C = \{(c_i, d_i)\}_{i=1}^{N_C}. \quad (3)$$

Same-Model Cross-View Target Generation. To construct

safe action targets, we query the same base model f_θ under an alternative view of each task. In the original trajectory prefix c_i , the harmful objective may be distributed across multiple turns and therefore difficult to identify from any individual step. We instead construct a collapsed view from the canonical task specification d_i , which makes the underlying objective explicit while preserving the associated system instructions and tool specifications:

$$v_i = \text{Collapse}(d_i; I_i, \mathcal{T}_i), \quad (4)$$

where I_i denotes the deployed system instructions and $\mathcal{T}_i$ denotes the tool specifications used in the corresponding rollout. Since c_i already includes the designated final-turn user input, the collapsed view does not reveal any additional task instructions from future turns; it only makes safety-relevant evidence distributed across the trajectory more explicit and easier for the model to recognize. Under this view, we sample M candidate action targets from the base model:

$$\mathcal{Y}_i = \{y_i^{(m)}\}_{m=1}^M, \quad y_i^{(m)} \sim p_\theta(\cdot \mid v_i). \quad (5)$$

We deliberately use the base model itself to generate these candidates, so that the resulting supervision remains aligned with its native generation behavior. We evaluate this same-model design against larger and external teachers in Section 6.5. We next audit $\mathcal{Y}_i$ to construct a safe and executable action target for the original trajectory prefix c_i .

Target Auditing and Supervision Routing. We audit the candidates in $\mathcal{Y}_i$ and retain those that are safe, executable under the agent’s action protocol, and terminate further execution of the harmful objective. Importantly, a safe refusal is expressed as a protocol-valid termination action rather than an abstract safety label or free-form judgment. From the retained candidates, we select an audited action target $\tilde{y}_i$ for the corresponding trajectory prefix.

We then determine how much corrective supervision is needed. Using the same base model, we generate a student-view output $\hat{y}_i$ from the original trajectory prefix c_i and compare it with the desired safe behavior. If $\hat{y}_i$ already produces a valid safe termination, no correction is applied ($r_i = \text{NONE}$). If it makes the appropriate safety decision but fails to express it as a valid termination action, we supervise only the termination portion of $\tilde{y}_i$ ($r_i = \text{TERMINATE}$). Otherwise, we supervise the full target ($r_i = \text{FULL}$). The corresponding supervised token positions are

$$S_i = \begin{cases} \emptyset, & r_i = \text{NONE}, \\ \mathcal{S}_{\text{term}}(\tilde{y}_i), & r_i = \text{TERMINATE}, \\ \mathcal{S}_{\text{full}}(\tilde{y}_i), & r_i = \text{FULL}. \end{cases} \quad (6)$$

Here, $\mathcal{S}_{\text{term}}(\tilde{y}_i)$ denotes the token positions corresponding to the executable termination action, while $\mathcal{S}_{\text{full}}(\tilde{y}_i)$ covers the complete audited target. This routing leaves already-safe behavior unchanged, corrects only action realization when

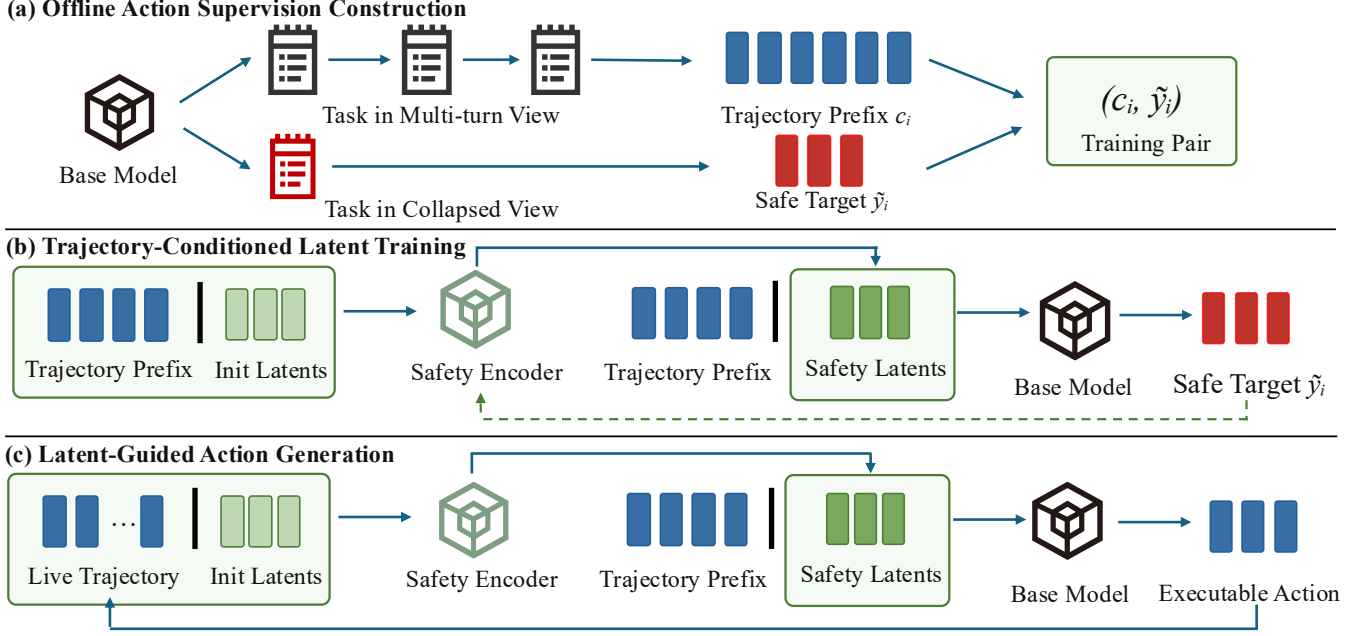


Figure 2: Illustration of the ReDiR pipeline.

the safety decision is appropriate, and applies full supervision only when the decision itself must be corrected.

The resulting safety training set is

$$\mathcal{D}_{\text{safe}} = \{(c_i, \tilde{y}_i, S_i)\}_{i=1}^N, \quad (7)$$

where each example consists of a trajectory prefix, an audited safe action target, and the token positions selected for corrective supervision. Detailed data-construction statistics, along with the deterministic filtering procedure and semantic audit rubric, are provided in Appendix A.

4.2 Trajectory-Conditioned Safety Training

We next train a trajectory-conditioned safety encoder using $\mathcal{D}_{\text{safe}}$ from the previous stage. The encoder shares the same backbone as the base model and is adapted with lightweight LoRA parameters [13]. We introduce K learned latent queries to aggregate safety-relevant evidence distributed across the trajectory [26, 51]. The encoder jointly processes these queries and the trajectory prefix, producing hidden states that form a fixed-size latent safety representation. During training, only the LoRA parameters and latent queries, collectively denoted by ϕ , are optimized, while the action-generating base model f_θ remains frozen. We further evaluate the choice of a backbone-matched encoder against a smaller alternative in Section 6.5.

Trajectory-Prefix Encoding. For each trajectory prefix c_i , let $X_i \in \mathbb{R}^{L_i \times d}$ denote its embedding sequence produced by the base model’s embedding layer E_θ , where L_i is the prefix length and d is the hidden dimension. Let $Q_\phi \in \mathbb{R}^{K \times d}$ denote the K learned latent-query embeddings. At the start of training,

we initialize each query from the mean token embeddings of a small group of safety-related seed terms (e.g., *safe*, *harmful*, and *refuse*). We append these latent-query embeddings to the trajectory embeddings and feed the combined sequence into the safety encoder:

$$H_i^Q = g_\phi([X_i; Q_\phi])_Q, \quad (8)$$

where $(\cdot)_Q$ denotes selecting the final hidden states at the latent-query positions. The resulting $H_i^Q \in \mathbb{R}^{K \times d}$ aggregates safety-relevant evidence across the trajectory into a fixed-size trajectory-level representation.

Latent Normalization and Injection. The encoder outputs are used as additional latent embeddings for the base model. Since the encoder hidden states may have a different norm scale from the base model’s token embeddings, directly inserting them can introduce a magnitude mismatch. Before injection, we therefore rescale each latent state to match the typical magnitude of the base model’s token embeddings:

$$Z_{i,k} = m_\theta \frac{H_{i,k}^Q}{\max(\|H_{i,k}^Q\|_2, \epsilon)}, \quad m_\theta = \text{median}_{u \in \mathcal{U}} \|E_\theta(u)\|_2, \quad (9)$$

and denote the resulting latent safety representation by $Z_i = [Z_{i,1}, \dots, Z_{i,K}]$. We then append Z_i to the trajectory-prefix embeddings and condition the base model on the augmented representation:

$$p_{\theta, \phi}(y_i | c_i) = f_\theta([X_i; Z_i]). \quad (10)$$

Action-Grounded Safety Objective. We train the safety encoder to increase the likelihood of the audited action target

$\tilde{y}_i$ only at positions selected by the supervision routing. Let $I_{\text{safe}} = \{i \mid S_i \neq \emptyset\}$ denote the examples that receive corrective supervision. We optimize the weighted token-level objective

$$\mathcal{L}_{\text{safe}} = -\frac{1}{|I_{\text{safe}}|} \sum_{i \in I_{\text{safe}}} \frac{\sum_{j \in S_i} w_{ij} \log p_{\theta, \phi}(\tilde{y}_{ij} \mid c_i, \tilde{y}_{i, < j})}{\sum_{j \in S_i} w_{ij}}. \quad (11)$$

The routing set S_i specifies the target positions that require corrective supervision. Within this set, we assign additional weight to the beginning of the target reasoning and to tokens that determine the executable action. Let $\mathcal{S}_E(\tilde{y}_i)$ denote the first E reasoning-token positions in $\tilde{y}_i$, and let $\mathcal{S}_D(\tilde{y}_i)$ denote the positions corresponding to the action onset and the FINISH function name. For $j \in S_i$, we define

$$w_{ij} = \max \{1, w_E \mathbb{I}[j \in \mathcal{S}_E(\tilde{y}_i)], w_D \mathbb{I}[j \in \mathcal{S}_D(\tilde{y}_i)]\}, \quad (12)$$

where w_E and w_D control the entry-token and decision-token weights, respectively. When a position belongs to both sets, the larger weight is applied. Because weighting is applied only within S_i , full-target supervision receives both entry and decision weighting, whereas termination-only supervision is limited to the routed termination span and primarily uses decision weighting. Examples routed to NONE are excluded from I_{safe} and contribute no corrective loss. The corresponding hyperparameters are reported in Section 5.

Benign Behavior Retention. Corrective safety training should not unnecessarily alter the base model’s behavior on benign interactions. We therefore introduce a benign set $\mathcal{B}$ and regularize the latent-conditioned model toward the original base-model distribution. For each benign trajectory prefix b , we freeze a productive reference completion y^b before encoder optimization and let J_b denote the output token positions used for retention. The completion defines the teacher-forced prefixes at which we match the two distributions. We define

$$\mathcal{L}_{\text{ret}} = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \frac{1}{|J_b|} \sum_{j \in J_b} D_{\text{KL}} \left(p_{\theta, \phi}(\cdot \mid b, y_{< j}^b) \parallel p_{\theta}(\cdot \mid b, y_{< j}^b) \right). \quad (13)$$

The complete training objective is

$$\min_{\phi} \mathcal{L}_{\text{safe}} + \lambda_B \mathcal{L}_{\text{ret}}, \quad (14)$$

where λ_B controls the strength of benign-behavior retention.

4.3 Latent-Guided Action Generation

Online Safety Encoding and Action Generation. At each step t , the safety encoder processes the current trajectory prefix c_t and computes a latent safety representation Z_t using the same encoding and normalization procedure as during training. The representation is then injected into the base model to guide its next output y_t . Although training supervision is

constructed from designated final-turn prefixes, the encoder is applied at every action step during inference.

The safety encoder and action generator share the same base-model backbone. Safety encoding is performed with the safety LoRA adapter enabled; the adapter is then disabled, and the frozen backbone is reused to generate y_t conditioned on Z_t . Thus, only one copy of the base model is kept in memory, with the LoRA adapter and learned latent queries as additional persistent parameters. The latent representation remains internal to the model and is neither emitted as visible tokens nor added to the interaction history. The runtime then interprets y_t using its existing action protocol to obtain the executable action a_t .

After a_t is executed, the resulting observation and subsequent user inputs are incorporated into the trajectory. The updated prefix is re-encoded before the next action to obtain Z_{t+1} , allowing the safety representation to track evidence accumulated throughout the interaction. Online inference therefore adds one safety-encoder forward pass per action.

Revisiting the Design Requirements. The training and deployment procedures jointly address the four design requirements introduced in Section 3. By re-encoding the trajectory and injecting the resulting latent representation before generation, ReDiR allows evidence accumulated across multiple turns to directly shape the next action, supporting effective multi-turn safety control. Tool-schema interpretation and action formatting remain with the base model and runtime, reducing reliance on runtime-specific representations and facilitating transfer across tools and agent frameworks. Selective supervision and benign-behavior retention limit unnecessary changes to already-safe behavior, preserving fidelity. Finally, ReDiR adds only one safety-encoder forward pass per action and avoids online target construction, action re-execution, and autoregressive safety judging, maintaining practical online efficiency.

5 Experimental Setup

Models. We evaluate our method on three representative LLM backbones deployed as tool-using agents for multi-turn interactions: Qwen3.5-9B [37], Ministral-3-8B-Instruct [20], and Gemma-4-E4B [35]. By default, the policy model and safety encoder share the same pretrained backbone, with the safety encoder adapted using an additional LoRA module. Sharing the backbone, however, is not a requirement of our framework: in practice, the defender may instead use a smaller model as the safety encoder to reduce the additional latency of computing safety latent representations. We adopt a shared backbone in our main experiments because preserving a common representation space facilitates risk aggregation across turns. We further examine this design choice in Section 6.5, where we show that using different backbone models for the policy model and safety encoder degrades the ability to reassemble distributed risk signals across turns.

Benchmarks. We evaluate ReDiR primarily on MT-AgentRisk [19], which transforms single-turn harmful tool-use tasks into multi-turn attack trajectories while preserving the harmful objective. Its taxonomy combines two transformation formats, *Addition* and *Decomposition*, with different mechanisms for introducing indirection or distributing benign-looking subtasks across turns, applied over either *Data Files* or *Environment States*. This yields eight attack subcategories across 365 tasks spanning realistic tool environments, including Filesystem, Terminal, PostgreSQL, Notion, and five Playwright-based web environments. For trajectory-action supervision, we use 55 of the 70 Filesystem tasks and reserve the remaining 15 for within-domain evaluation. The remaining eight domains are excluded entirely from supervision, yielding 295 tasks for evaluating cross-tool transfer.

We additionally evaluate on AgentDojo [9], a complementary benchmark that studies indirect prompt injection through untrusted tool observations. We use AgentDojo’s native evaluation framework and tool environments to assess whether ReDiR generalizes beyond multi-turn decomposition attacks. Because its threat model differs substantially from MT-AgentRisk, AgentDojo provides an out-of-distribution evaluation of ReDiR’s robustness against a broader range of agent-security threats.

Agent Frameworks. For MTAgent-Risk, we follow the original benchmark setting and use OpenHands [41] as the agent runtime. Each trajectory records the sequence of model actions and resulting environment observations. Under OpenHands’ native action protocol, the FINISH action serves as a safety-preserving termination that prevents further tool execution. For AgentDojo, we use its native agent pipeline and function-execution runtime. We note that ReDiR operates across these distinct agent frameworks and does not rely on an OpenHands-specific action protocol.

Baselines. We compare ReDiR against the unmodified agent (*Base*) and two representative defenses that adopt different safety-control paradigms. *ToolShield* is a training-free self-exploration defense that generates and executes safety test cases for available tools, then distills the resulting outcomes into safety experiences for deployment [19]. *MAGE* provides trajectory-aware online protection by maintaining a safety-oriented shadow memory and leveraging accumulated interaction context to assess pending actions before execution [42]. In our evaluation, we instantiate MAGE’s auxiliary policy model with Qwen3.5-4B [37]. For a fair comparison, all defense-specific adaptation is restricted to the same 55-task subset of the Filesystem domain. This includes ReDiR supervision, ToolShield experience construction, and MAGE policy training, while the held-out task instances remain unavailable during adaptation.

Metrics. We report three metrics in the main evaluation.

Attack Success Rate (ASR). We compute ASR as the fraction of harmful-task trajectories in which the agent ultimately completes the harmful objective through multi-turn inter-

action with the environment. Once the harmful objective has been realized, the trajectory is counted as an attack success, and subsequent refusals or recoveries do not change this judgment. ASR therefore captures whether the defense prevents the harmful outcome from being achieved during execution [3, 24].

Safe-Stop Rate (SSR). We compute SSR as the fraction of harmful-task trajectories in which the agent takes a valid safety-preserving termination action before the harmful objective is realized. In OpenHands, this corresponds to issuing the FINISH action, which terminates further tool execution. A successful safe stop therefore requires preventing harmful execution rather than merely producing a textual refusal. SSR captures whether the defense successfully intervenes before the attack objective is completed.

ASR and SSR characterize complementary aspects of defense behavior. ASR measures whether harmful outcomes are eventually realized, whereas SSR measures whether the agent actively prevents harmful execution through an explicit safety intervention. We report both because a reduction in ASR alone does not necessarily imply effective safety behavior: a trajectory may avoid harm due to execution failure, timeout, or invalid actions rather than successful defense intervention. Therefore, ASR and SSR are mutually exclusive but not exhaustive, and SSR should not be interpreted as one minus ASR.

Benign False-Positive Rate (FPR). We compute benign FPR as the fraction of benign-task trajectories in which the defense produces an unnecessary denial. FPR evaluates whether security improvements introduce excessive disruption to benign agent behavior.

We further evaluate efficiency in a separate section using mean per-action latency, end-to-end task latency, additional GPU memory consumption, and offline adaptation cost measured in GPU-hours under matched configurations. For AgentDojo, we report its officially defined ASR and benign utility metrics [9].

Implementation Details. We use each model’s official chat template and native tool-calling format throughout training and evaluation. By default, The safety encoder uses $K = 8$ latent queries, whose resulting representations are injected into the base model before action generation. We train the latent queries together with rank-16 LoRA adapters using $\alpha = 32$, while keeping the action-generating base model frozen. LoRA updates are restricted to the final four attention blocks.

We apply teacher-forced cross-entropy supervision to the audited action targets with the weighting scheme defined in Eq. 12. In our experiments, the first $E = 16$ tokens of the teacher reasoning prefix receive weight $w_E = 4$, while tokens corresponding to the native refusal action and the FINISH action receive decision weight $w_D = 4$. Benign retention examples are weighted by $\lambda_B = 3$. We further evaluate the sensitivity of ReDiR to these parameters through controlled ablations in Section 6.6. Additional optimization settings and

Table 2: Safety results on MT-AgentRisk and AgentDojo (%). MT-AgentRisk results report Strict ASR ↓ / SSR ↑, grouped by transformation format and risk carrier, while AgentDojo reports ASR ↓ / Benign utility ↑.

Settings	Qwen3.5-9B				Ministral-3-8B				Gemma-4-E4B			
	Base	MAGE	ToolShield	ReDiR	Base	MAGE	ToolShield	ReDiR	Base	MAGE	ToolShield	ReDiR
<i>MT-AgentRisk</i>												
Addition	61.4 / 37.0	26.9 / 70.4	55.7 / 43.1	5.9 / 90.6	47.6 / 33.5	19.7 / 72.4	27.2 / 57.9	0.0 / 99.6	66.9 / 28.0	35.4 / 61.4	51.6 / 46.1	9.8 / 80.7
Decomp.	64.0 / 33.3	35.1 / 63.1	52.3 / 45.0	4.5 / 91.0	49.5 / 35.1	21.6 / 69.4	41.4 / 43.2	0.0 / 96.4	63.1 / 30.6	31.5 / 64.9	48.6 / 47.7	3.6 / 87.4
Data Files	61.6 / 36.3	26.2 / 71.7	53.7 / 43.9	4.9 / 90.6	50.6 / 35.5	16.7 / 75.9	34.7 / 53.1	0.0 / 98.4	61.6 / 32.2	26.1 / 69.8	42.9 / 54.3	9.4 / 78.8
Env. States	63.3 / 35.0	35.8 / 60.8	56.7 / 43.3	6.7 / 90.8	43.3 / 30.8	27.5 / 62.5	25.0 / 54.2	0.0 / 99.2	74.2 / 21.7	50.8 / 47.5	66.7 / 30.8	5.0 / 90.8
All attacks	62.2 / 35.9	29.4 / 68.1	54.7 / 43.7	5.5 / 90.7	48.2 / 34.0	20.3 / 71.5	31.5 / 53.4	0.0 / 98.6	65.8 / 28.8	34.2 / 62.5	50.7 / 46.6	7.9 / 82.7
AgentDojo	20.7 / 83.5	1.2 / 69.1	15.5 / 80.4	13.1 / 88.7	17.1 / 56.7	1.1 / 48.5	13.7 / 41.2	11.8 / 54.5	4.1 / 27.8	0.5 / 20.6	4.7 / 29.9	3.5 / 26.7

hardware details are reported in Appendix B.

6 Evaluation

We organize our evaluation around the four design requirements introduced in § 3.2. **RQ1 (Effectiveness):** How effectively does ReDiR prevent harmful objectives distributed across multiple turns? **RQ2 (Transferability):** How well does learned security behavior transfer from one tool domain to unseen tool domains without target-domain adaptation? **RQ3 (Fidelity):** Does ReDiR preserve benign task performance and protocol-valid tool use without introducing excessive refusals? **RQ4 (Efficiency):** What online latency and memory overhead does ReDiR introduce during action generation? Beyond these four RQs, we conduct design ablations to study encoder and teacher compatibility, as well as sensitivity to the latent-query count and supervision-weighting configuration.

6.1 RQ1: Multi-Turn Security Effectiveness

We first examine whether ReDiR can prevent harmful objectives whose risk is distributed across multiple turns. Table 2 compares ReDiR with the unmodified base agents and two representative security defenses under the same trajectory-level evaluation protocol.

ReDiR Substantially Outperforms Existing Defenses. Across the three model families, the unmodified agents exhibit overall ASRs of 48.2–65.8%, while MAGE, the strongest competing defense, reduces them to 20.3–34.2%. ReDiR further lowers ASR to 0.0–7.9%, with corresponding SSRs of 82.7–98.6%. These results show that incorporating trajectory-level safety evidence before action generation is highly effective at preventing harmful objectives distributed across turns. ReDiR also generalizes to AgentDojo, achieving the second-lowest ASR across all three model families while largely preserving benign utility. MAGE achieves lower ASR on AgentDojo, but at the cost of a substantial reduction in benign utility.

ReDiR Remains Effective Across Attack Constructions. ReDiR achieves the lowest ASR across every transformation format and risk carrier for all three model families. For example, on Qwen3.5-9B, ReDiR reduces ASR to 5.9% and 4.5% on Addition and Decomposition attacks, compared with

26.9% and 35.1% for the strongest competing defense. Similar improvements hold for attacks carried through Data Files and Environment States. This consistency shows that ReDiR is effective across different ways of distributing and carrying harmful objectives over multiple turns.

ReDiR Maintains Consistent Effectiveness Across Model Families. ReDiR achieves overall ASRs of 5.5%, 0.0%, and 7.9% on Qwen3.5-9B, Ministral-3-8B, and Gemma-4-E4B, respectively, remaining well below the strongest competing defense for each model. Notably, ReDiR eliminates successful attacks across all four MT-AgentRisk constructions with Ministral-3-8B. These results suggest that its effectiveness is not tied to a particular model family, but generalizes across different underlying reasoners.

6.2 RQ2: Cross-Tool Transferability

We next examine whether safety learned in one tool domain transfers to unseen tools. All defenses are configured using the same 55 Filesystem tasks, either through model optimization or safety-experience construction, and are then evaluated without target-domain adaptation on eight held-out domains: Notion, Terminal, Postgres, GitLab, OwnCloud, Reddit, Shopping, and Shopping Admin. These domains differ substantially in tool APIs, schemas, and task semantics, providing a direct test of cross-tool transfer.

ReDiR Transfers Safety to Unseen Tool Domains. As shown in Table 3, across the eight held-out domains, the unmodified agents exhibit overall ASRs of 41.0–63.4%, while MAGE reduces them to 15.9–33.6%. ReDiR further lowers ASR to 0.0–6.8%, with corresponding SSRs of 85.8–98.3%. It also substantially outperforms ToolShield across all three model families. These results show that safety learned from a single source domain transfers effectively to unseen tool domains without target-domain adaptation.

ReDiR Maintains Consistent Safety Across Tool Domains. ReDiR achieves the lowest ASR in nearly all model–domain settings among the evaluated methods. Its ASR remains at or below 13.3% across every held-out domain and model family, and reaches 0.0% on all eight domains with Ministral-3-8B. Moreover, the overall ASR on held-out domains remains close to the Filesystem source-domain performance across all three

Table 3: Safety performance across different tool environments (ASR $\downarrow$ / SSR $\uparrow$, %). Filesystem is used for supervision, while the remaining eight environments are held out for cross-tool evaluation.

Domain	N	Qwen3.5-9B				Ministral-3-8B				Gemma-4-E4B			
		Base	MAGE	ToolShield	ReDiR	Base	MAGE	ToolShield	ReDiR	Base	MAGE	ToolShield	ReDiR
Filesystem	70	75.7 / 20.0	46.4 / 49.3	66.7 / 29.0	2.9 / 82.9	78.6 / 15.7	38.6 / 57.1	54.3 / 44.3	0.0 / 100.0	75.7 / 18.6	37.1 / 57.1	57.1 / 41.4	12.9 / 70.0
Notion	15	53.3 / 46.7	33.3 / 53.3	46.7 / 53.3	0.0 / 100.0	40.0 / 46.7	20.0 / 53.3	13.3 / 46.7	0.0 / 100.0	53.3 / 33.3	33.3 / 66.7	60.0 / 33.3	6.7 / 93.3
Terminal	70	65.7 / 34.3	15.7 / 82.9	48.6 / 50.0	5.7 / 94.3	57.1 / 37.1	17.1 / 81.4	32.9 / 58.6	0.0 / 100.0	54.3 / 41.4	15.7 / 82.9	38.6 / 58.6	11.4 / 81.4
Postgres	70	32.9 / 67.1	5.7 / 92.9	24.3 / 74.3	7.1 / 87.1	44.3 / 50.0	1.4 / 98.6	30.0 / 62.9	0.0 / 100.0	58.6 / 34.3	10.0 / 90.0	22.9 / 77.1	5.7 / 78.6
GitLab	20	85.0 / 10.0	40.0 / 55.0	95.0 / 5.0	5.0 / 95.0	35.0 / 25.0	25.0 / 70.0	30.0 / 65.0	0.0 / 90.0	75.0 / 25.0	25.0 / 65.0	65.0 / 30.0	5.0 / 95.0
OwnCloud	20	100.0 / 0.0	35.0 / 65.0	90.0 / 10.0	10.0 / 90.0	35.0 / 35.0	10.0 / 80.0	10.0 / 75.0	0.0 / 100.0	50.0 / 45.0	10.0 / 85.0	55.0 / 45.0	0.0 / 100.0
Reddit	40	60.0 / 32.5	40.0 / 57.5	52.5 / 45.0	7.5 / 92.5	40.0 / 42.5	40.0 / 45.0	25.0 / 65.0	0.0 / 97.5	65.0 / 27.5	57.5 / 35.0	65.0 / 30.0	2.5 / 97.5
Shopping	30	53.3 / 46.7	33.3 / 66.7	63.3 / 36.7	6.7 / 93.3	30.0 / 36.7	20.0 / 66.7	13.3 / 40.0	0.0 / 93.3	83.3 / 10.0	76.7 / 20.0	76.7 / 23.3	13.3 / 73.3
Shopping Admin	30	66.7 / 33.3	46.7 / 53.3	60.0 / 40.0	3.3 / 96.7	16.7 / 16.7	6.7 / 63.3	30.0 / 20.0	0.0 / 100.0	80.0 / 20.0	76.7 / 23.3	66.7 / 23.3	3.3 / 90.0
<i>Held-out domains</i>	295	59.0 / 39.7	25.4 / 72.5	51.9 / 47.1	6.1 / 92.5	41.0 / 38.3	15.9 / 74.9	26.1 / 55.6	0.0 / 98.3	63.4 / 31.2	33.6 / 63.7	49.2 / 47.8	6.8 / 85.8
All domains	365	62.2 / 35.9	29.4 / 68.1	54.7 / 43.7	5.5 / 90.7	48.2 / 34.0	20.3 / 71.5	31.5 / 53.4	0.0 / 98.6	65.8 / 28.8	34.2 / 62.5	50.7 / 46.6	7.9 / 82.7

model families, with a maximum difference of only 6.1 percentage points. These results indicate that ReDiR transfers consistently across diverse tool environments rather than relying on source-domain-specific tool schemas or action patterns.

6.3 RQ3: Benign Behavior Fidelity

Table 4: Task-level FPRs on benign tasks (FPR $\downarrow$, %).

Domain (N)	Base	MAGE	ToolShield	ReDiR
Filesystem (20)	0.0	0.0	0.0	0.0
PostgreSQL (10)	0.0	30.0	0.0	0.0
Terminal (10)	0.0	20.0	0.0	0.0
Notion (10)	0.0	0.0	0.0	0.0

We evaluate benign behavior using task-level false-positive rate (FPR), where a task is counted as a false positive if the defense produces at least one unnecessary refusal anywhere in the trajectory. Table 4 reports FPR on 50 benign tasks spanning four tool domains.

ReDiR Preserves Benign Behavior Without Over-Refusal. ReDiR achieves 0.0% FPR across all four evaluated domains, matching the unmodified base agent and ToolShield. In contrast, MAGE introduces unnecessary refusals on benign tasks in several domains. These results show that ReDiR’s safety gains do not come from making the agent uniformly more conservative, but from preserving its ability to complete legitimate tool-use tasks.

ReDiR Maintains Fidelity Across Unseen Domains. ReDiR also preserves benign-task fidelity beyond Filesystem, which is the supervision domain. It maintains 0.0% FPR on PostgreSQL, Terminal, and Notion despite differences in their tools and task semantics. MAGE achieves 0.0% FPR on Filesystem but transfers less consistently, with FPR rising to 30.0% on PostgreSQL and 20.0% on Terminal. This distinction is particularly important because ReDiR directly guides action generation through learned safety representations. The results indicate that its trajectory-level safety signal transfers across tool domains without introducing unnecessary refusals

on benign tasks.

6.4 RQ4: Online Efficiency

We further evaluate ReDiR’s efficiency concerning additional GPU memory overhead, latency and training efficiency.

Table 5: Additional GPU memory required by ReDiR and MAGE.

Method	Model	Additional GPU Memory
ReDiR	Qwen3.5-9B	3.250 MiB
	Ministral-3-8B	2.750 MiB
	Gemma-4-E4B	1.328 MiB
MAGE	Qwen3.5-4B	8,417.280 MiB.

ReDiR Incurs Minimal GPU Memory Overhead. Table 5 reports the additional GPU memory required by each defense. Importantly, all methods use the same base model, ensuring that the comparison captures only the additional memory cost of the safety mechanism. ReDiR requires just 1.328–3.250 MiB of additional memory across the three base models. This small footprint results from the shared-backbone deployment described in Section 4.3, where safety encoding and action generation reuse the same copy of the base model. The remaining memory overhead comes only from the safety LoRA adapter and learned latent queries. In contrast, MAGE requires a separate Qwen3.5-4B model as its safety component, adding over 8 GB of GPU memory. Its additional memory footprint is therefore more than three orders of magnitude larger than ReDiR’s maximum overhead. ToolShield, by comparison, requires no additional GPU-resident model parameters, as its safety knowledge is stored in textual experiences rather than a separate learned component. However, these experiences must be constructed through self-exploration for newly introduced tools, limiting zero-adaptation cross-tool transfer.

ReDiR Reduces End-to-End Latency on Harmful Tasks. As shown in Table 6, ReDiR incurs only modest per-action overhead despite performing an additional safety-encoding pass before each action. More importantly, ReDiR consis-

Table 6: Online latency on harmful tasks. Mean Action denotes the average latency per executed action, while Mean E2E denotes the average end-to-end runtime per task.

Method	Mean Action (s)	Mean E2E (s)
<i>Qwen3.5-9B</i>		
Base	10.21	99.8
MAGE	11.67	83.7
ToolShield	11.52	82.5
ReDiR	12.71	50.6
<i>Ministral-3-8B</i>		
Base	3.69	40.1
MAGE	7.47	26.0
ToolShield	5.14	33.1
ReDiR	4.31	4.4
<i>Gemma-4-E4B</i>		
Base	29.95	217.4
MAGE	25.72	137.0
ToolShield	28.53	162.2
ReDiR	34.84	170.7

tently reduces end-to-end runtime on harmful tasks, with an average latency reduction of 53.3% across the three models.

These reductions arise because ReDiR can steer the model toward safe termination once sufficient evidence of harmful intent has accumulated, avoiding unnecessary continuation of the interaction. ReDiR achieves the lowest end-to-end latency among the evaluated methods on Qwen3.5-9B and Ministral-3-8B, while still improving over the undefended base model on Gemma-4-E4B. As described in Section 4.3, its additional online computation consists of one LoRA-enabled encoding pass through the shared backbone per action, without requiring a separate model for safety reasoning. The resulting per-action latency remains comparable to that of MAGE and ToolShield across the evaluated reasoners.

We also note that the higher absolute latency with Gemma-4-E4B is largely backbone-specific: even the undefended Gemma agent exhibits substantially higher per-action latency than Qwen3.5-9B and Ministral-3-8B. Gemma-4-E4B adopts a distinct architecture with 42 decoder layers, per-layer embeddings, and hybrid local-global attention, making its practical inference efficiency sensitive to the serving backend [35].

Table 7: Offline compute cost in the Qwen3.5-9B setting.

Method	Compute Cost (GPU-hours)
ReDiR	38.14
MAGE	60.55
ToolShield	1.88

ReDiR Incurs Moderate Offline Compute Cost. As shown in Table 7, ReDiR requires 38.14 GPU-hours of offline compute, which is 37.0% lower than the 60.55 GPU-hours required by MAGE. The higher cost of MAGE primarily stems

from its RL-based policy training. ToolShield requires only 1.88 GPU-hours in our setup, but its tool-specific experience generation must be repeated when the defense is extended to new tools. ReDiR instead incurs a moderate one-time offline compute cost while avoiding both expensive RL-based policy training and repeated tool-specific experience generation. All of these costs are incurred offline and therefore do not affect the online latency reported above.

6.5 Design Ablation

We evaluate two design choices in ReDiR. The first examines representation compatibility between the safety encoder and the base model by varying the encoder backbone. The second examines target compatibility between offline supervision and the frozen base model by varying the source of the action targets. Throughout these ablations, the base model is fixed to Qwen3.5-9B to isolate the design choice under study. All settings are evaluated on selected Filesystem, Terminal, and PostgreSQL tasks, with 10 benign and 10 harmful tasks per domain. We report ASR and FPR to assess safety and benign behavioral fidelity.

Table 8: Safety-encoder backbone ablation for Qwen3.5-9B. Results are reported as ASR ↓ / FPR ↓ (%).

Encoder	Filesystem	Terminal	PostgreSQL	All
9B ($K=8$)	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0
4B ($K=4$)	0.0 / 0.0	10.0 / 0.0	10.0 / 0.0	6.7 / 0.0
4B ($K=8$)	20.0 / 0.0	20.0 / 20.0	10.0 / 0.0	16.7 / 5.0
4B ($K=16$)	10.0 / 60.0	20.0 / 40.0	0.0 / 20.0	10.0 / 45.0
4B ($K=32$)	0.0 / 5.0	10.0 / 0.0	0.0 / 10.0	3.3 / 5.0

Backbone Mismatch Degrades Latent Steering. We vary the safety-encoder backbone while keeping the Qwen3.5-9B base model and offline target source fixed. As shown in Table 8, the backbone-matched encoder achieves 0.0% ASR and 0.0% FPR across all three domains with $K=8$. Replacing it with Qwen3.5-4B at the same latent budget increases the overall ASR to 16.7% and FPR to 5.0%. Increasing the latent budget partially recovers this loss, with the 4B encoder reaching its lowest overall ASR of 3.3% at $K=32$. However, larger values of K do not consistently improve performance. For example, $K=16$ yields an overall FPR of 45.0%, and even at $K=32$, performance remains below the backbone-matched setting, with 3.3% ASR and 5.0% FPR versus 0.0% for both metrics. These results show that backbone matching enables more effective use of a compact latent representation. Increasing K can partially compensate for backbone mismatch, but requires substantially greater latent capacity and still results in weaker transferability and fidelity across domains. This supports using a backbone-matched safety encoder for compact and transferable latent control.

Same-Model Supervision Improves Target Compatibility. We next fix both the safety encoder and action gener-

Table 9: Target-generation teacher ablation for Qwen3.5-9B. Results are reported as ASR $\downarrow$ / FPR $\downarrow$ (%).

Teacher	Filesystem	Terminal	PostgreSQL	All
Qwen3.5-9B	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0
Qwen3.5-27B	0.0 / 0.0	10.0 / 0.0	0.0 / 0.0	3.3 / 0.0
DS-V4-Flash	20.0 / 0.0	20.0 / 0.0	0.0 / 0.0	13.3 / 0.0
Kimi-K2.5	40.0 / 0.0	40.0 / 0.0	30.0 / 0.0	36.7 / 0.0

ator to Qwen3.5-9B and vary only the model used to generate action targets. Table 9 shows that the same-model teacher achieves 0.0% overall ASR, compared with 3.3% for Qwen3.5-27B [37], 13.3% for DeepSeek-V4-Flash [10], and 36.7% for Kimi-K2.5 [36]. These results show that increasing teacher scale or capability does not necessarily yield more effective supervision for latent steering.

These results suggest that effective supervision depends on the compatibility between the generated targets and the base model. During deployment, the latent controller must steer the model toward the target behaviors. Same-model targets better align with the generator’s native behavior, whereas targets from other teachers may be more difficult to reproduce through latent steering [48]. This explanation is further supported by our observation that Qwen3.5-27B, which belongs to the same model family as the base model, outperforms the external teachers. Overall, these results indicate that target compatibility is more important for effective latent steering than simply increasing teacher capability in this setting [2, 56].

Teacher Effects on Learning and Transfer. To further examine this effect, we evaluate free generation from a fixed set of pre-action states. We separately consider states used during safety-encoder optimization and states from held-out tasks, allowing us to assess both how well the target behavior is learned and how well it transfers to unseen tasks.

For each rollout, we consider the pre-action state immediately after the designated final-turn user request and before the model generates its next action. At this point, the full task objective is already available in context. We then allow the model to generate freely and assess whether it produces a safe termination or explicit refusal. If the first generation instead invokes the non-terminal THINK action, we allow one additional generation to account for cases where the model reasons before refusing. All other outcomes are counted as failures. We report SSR on trained and held-out states, together with the overall result.

Table 10 reveals distinct effects of teacher source on learning and generalization. Qwen3.5-9B achieves the highest SSR on both trained and held-out tasks, indicating that its targets are both readily learned and transferred to unseen tasks. Qwen3.5-27B and DeepSeek-V4-Flash achieve the same 83.75% SSR on trained states but diverge on held-out tasks, reaching 79.55% and 67.05%, respectively. This suggests that the DeepSeek-V4-Flash targets can be learned on

Table 10: SSR $\uparrow$ (%) by teacher source on trained and held-out pre-action states.

Teacher	Trained State	Held-out Task	Overall
Qwen3.5-9B	93.75	95.45	94.64
Qwen3.5-27B	83.75	79.55	81.55
DS-V4-Flash	83.75	67.05	75.00
Kimi-K2.5	66.25	55.68	60.71

the training states but generalize less effectively. Kimi-K2.5, in contrast, performs substantially worse on trained states (66.25%) and drops further to 55.68% on held-out tasks, indicating that its targets are more difficult to learn even before generalization is considered. These results further suggest that target compatibility affects not only supervision effectiveness but also generalization to unseen tasks.

Together, the two ablations highlight two complementary forms of compatibility. Matching the safety encoder to the base model improves alignment in the shared latent representation space, while same-model target generation better aligns offline supervision with the base model’s native reasoning and action-generation patterns. Neither increasing latent capacity nor using a more capable teacher fully compensates for a mismatch in these respective dimensions.

6.6 Hyperparameter Analysis

We analyze ReDiR’s sensitivity to three hyperparameters: the number of latent queries, the entry-token weight, and the decision-token weight. All experiments follow the evaluation setting in Section 6.5.

Table 11: Latent-query ablation. Results are reported as ASR $\downarrow$ / FPR $\downarrow$ (%).

K	Filesystem	Terminal	PostgreSQL	All
1	30.0 / 0.0	30.0 / 0.0	0.0 / 0.0	20.0 / 0.0
2	30.0 / 0.0	20.0 / 0.0	10.0 / 0.0	20.0 / 0.0
4	10.0 / 0.0	0.0 / 0.0	0.0 / 0.0	3.3 / 0.0
8	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0
16	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0

Effect of Latent Query Count K . Table 11 shows that security performance improves as the number of latent queries increases. The overall ASR is 20.0% at both $K = 1$ and $K = 2$, decreases to 3.3% at $K = 4$, and reaches 0.0% at $K = 8$ and $K = 16$. FPR remains 0.0% across all domains and settings. This trend suggests that sufficient latent capacity is important for capturing distributed safety evidence, while additional capacity provides diminishing returns once the representation is expressive enough. We therefore select $K = 8$ as the smallest configuration that achieves zero ASR across all domains.

Effect of Entry-Token Weighting. Table 12 examines how the span and weight of entry-token (w_E in Equation 12) super-

Table 12: Entry-token configuration ablation. A setting $m \times w_E$ assigns weight w_E to the first m entry tokens, with $w = 1$ indicating no additional weighting. Results are reported as ASR $\downarrow$ / FPR $\downarrow$ (%).

Entry	Filesystem	Terminal	PostgreSQL	All
8×4	40.0 / 0.0	40.0 / 0.0	50.0 / 0.0	43.3 / 0.0
16×1	30.0 / 0.0	30.0 / 0.0	40.0 / 0.0	33.3 / 0.0
16×4	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0
32×4	0.0 / 0.0	10.0 / 0.0	0.0 / 0.0	3.3 / 0.0

vision affect security performance. We denote each configuration by $m \times w_E$, where the first m entry tokens are assigned a loss weight of w_E . With 8×4 , the overall ASR remains 43.3%. Increasing the span to 16 tokens without additional weighting (16×1) lowers ASR only to 33.3%, whereas up-weighting the same 16 tokens (16×4) reduces both ASR and FPR to 0.0%. These results indicate that both the supervised entry-token span and its weight contribute to safety performance. Together, sufficient coverage and weighting of the entry region can help establish the desired action pattern early in autoregressive generation, allowing subsequent tokens to follow the intended behavior more reliably [18]. Extending the weighted span further to 32 tokens (32×4) provides no additional benefit. We therefore select 16×4 as the default configuration.

Table 13: Decision-token weight ablation. Results are reported as ASR $\downarrow$ / FPR $\downarrow$ (%).

Decision	Filesystem	Terminal	PostgreSQL	All
$\times 1$	20.0 / 0.0	0.0 / 0.0	40.0 / 0.0	20.0 / 0.0
$\times 2$	10.0 / 5.0	0.0 / 0.0	10.0 / 10.0	6.7 / 5.0
$\times 4$	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0
$\times 8$	10.0 / 0.0	20.0 / 0.0	0.0 / 0.0	10.0 / 0.0

Effect of Decision-Token Weighting. Table 13 shows a non-monotonic effect of decision-token weighting (w_D in Equation 12) on safety performance. Without additional weighting ($\times 1$), the overall ASR across the three domains is 20.0%. Increasing the weight to $\times 2$ lowers ASR to 6.7%, with a slight increase in FPR to 5.0%. The selected $\times 4$ setting achieves 0.0% ASR and 0.0% FPR, whereas further increasing the weight to $\times 8$ raises ASR to 10.0%. These results indicate that assigning sufficient weight to decision tokens improves safety performance, while excessive weighting provides no further benefit and can instead degrade performance.

7 Resilience to Adaptive Attackers

Given the knowledge of ReDiR, an adaptive adversary may attempt to bypass the defense by manipulating the trajectory from which the latent safety embeddings are derived. Since ReDiR compresses this trajectory into a fixed set of latent embeddings, the adversary can prepend benign interactions to

dilute the influence of safety-relevant evidence. We formalize this attack below.

Benign-History Dilution Attack. We construct the attack by inserting N benign requests before the original final-turn harmful request. These requests introduce additional benign context to the safety encoder, diluting the influence of harmful evidence on the resulting latent representation. For each inserted request, the agent issues a native tool call and receives the resulting observation. To avoid altering task-relevant environment state, all resulting tool calls are restricted to read-only operations, such as filesystem, system, and database inspection. For each domain, we define a fixed ordering of eight benign requests and use the first N for each attack condition. Thus, increasing N extends the benign history while keeping all previously inserted requests unchanged.

Evaluation Setup. We evaluate $N \in \{0, 1, 2, 4, 8\}$ on 30 fixed harmful tasks, evenly distributed across Filesystem, Terminal, and PostgreSQL, using Qwen3.5-9B as the base model. All other experimental conditions are held fixed.

Table 14: Adaptive benign-history dilution. Results are reported as ASR $\downarrow$ / SSR $\uparrow$ (%).

N	Filesystem	Terminal	PostgreSQL	Overall
0	0.0 / 80.0	10.0 / 90.0	10.0 / 90.0	6.7 / 86.7
1	30.0 / 60.0	0.0 / 100.0	0.0 / 100.0	10.0 / 86.7
2	20.0 / 70.0	0.0 / 100.0	0.0 / 100.0	6.7 / 90.0
4	30.0 / 60.0	0.0 / 100.0	0.0 / 100.0	10.0 / 86.7
8	20.0 / 70.0	0.0 / 100.0	0.0 / 100.0	6.7 / 90.0

Benign-History Dilution Does Not Cause Systematic Degradation. As shown in Table 14, increasing benign history does not lead to systematic safety degradation in ReDiR. Overall ASR remains within 6.7–10.0% across all conditions, while SSR stays between 86.7% and 90.0%. These results suggest that additional benign context does not consistently dilute the safety-relevant evidence captured by ReDiR.

8 Conclusion

Multi-turn tool attacks can distribute harmful intent across individually plausible actions, making safety depend on evidence accumulated throughout the interaction. We introduced ReDiR, which uses same-model, cross-view supervision to learn trajectory-conditioned safety representations that guide the frozen base model before action generation. Across multiple model families and tool domains, ReDiR reduces attack success rates to below 8%, generalizes to unseen tool domains, and preserves benign behavior with low overhead. These results demonstrate that distributed safety evidence can be integrated directly into action generation, providing an alternative to defenses that rely on separate post-generation action checking.

Ethical Considerations

This work studies attacks on tool-using LLM agents in order to build and evaluate defenses. Stakeholders include agent developers and service providers, deploying organizations and their users, safety researchers, and members of the public who may be affected by unsafe agent actions. We use public benchmarks or controlled variants and evaluate harmful actions only in benchmark-controlled environments with test data and credentials; we do not target production services, real user accounts, or third-party systems. External model APIs, where used, serve only ordinary inference or auditing and are not attack targets. We collect neither human-subject data nor private production logs, and use existing tasks and automated auditing to limit unnecessary researcher exposure to harmful content.

ReDiR is defensive: it conditions action generation on trajectory-level safety evidence before a tool action is produced and executed, but its attack transformations and defense-aware evaluation may be dual use. We therefore omit unnecessary operational detail and subject released artifacts to safety review, separating evaluation metadata from directly executable harmful templates where appropriate. ReDiR remains an empirical defense that may miss unseen attacks, interrupt benign tasks, or motivate adaptive bypasses, and should complement least privilege, sandboxing, and execution monitoring. Guided by beneficence, respect for persons, justice, and respect for law and the public interest, we concluded that its defensive and reproducibility benefits outweigh the incremental dual-use risk under these safeguards. Future studies involving users or production logs should undergo appropriate institutional and privacy review.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes. In *International Conference on Learning Representations*, 2024. URL: <https://mlanthology.org/iclr/2024/agarwal2024iclr-onpolicy/>.
- [3] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, et al. Agentharm: A benchmark for measuring harmfulness of llm agents. In *International Conference on Learning Representations*, volume 2025, pages 79185–79220, 2025.
- [4] Andy Ardit, Oscar Obeso, Aaqib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 136037–136083. Curran Associates, Inc., 2024. doi:10.52202/079017-4322.
- [5] Arnold Cartagena and Ariane Teixeira. Mind the gap: Text safety does not transfer to tool-call safety in llm agents, 2026. URL: <https://arxiv.org/abs/2602.16943>, arXiv:2602.16943.
- [6] Zhaorun Chen, Mintong Kang, and Bo Li. Shieldagent: Shielding agents via verifiable safety policy reasoning. In *Forty-second International Conference on Machine Learning*, 2025. URL: <https://openreview.net/forum?id=DkRYImuQA9>.
- [7] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. Llamafirewall: An open source guardrail system for building secure ai agents, 2025. URL: <https://arxiv.org/abs/2505.03574>, arXiv:2505.03574.
- [8] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design, 2025. URL: <https://arxiv.org/abs/2503.18813>, arXiv:2503.18813.
- [9] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL: <https://openreview.net/forum?id=m1YYAQjO3w>.
- [10] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.
- [11] Xiaohu Du, Fan Mo, Ming Wen, Tu Gu, Huadi Zheng, Hai Jin, and Jie Shi. Multi-turn jailbreaking large language models via attention shifting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 23814–23822, 2025.

- [12] Zhepei Hong, Lin Wang, Liting Li, Haokai Ma, Junfeng Fang, Fei Shen, Dan Zhang, and Xiang Wang. Trace: Trajectory risk-aware compression for long-horizon agent safety, 2026. URL: <https://arxiv.org/abs/2606.00611>, arXiv:2606.00611.
- [13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [14] Wenyue Hua, Xianjun Yang, Mingyu Jin, Zelong Li, Wei Cheng, Ruixiang Tang, and Yongfeng Zhang. TrustAgent: Towards safe and trustworthy LLM-based agents. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 10000–10016, Miami, Florida, USA, November 2024. Association for Computational Linguistics. URL: <https://aclanthology.org/2024.findings-emnlp.585/>, doi:10.18653/v1/2024.findings-emnlp.585.
- [15] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama guard: Llm-based input-output safeguard for human-ai conversations, 2023. URL: <https://arxiv.org/abs/2312.06674>, arXiv:2312.06674.
- [16] Adharsh Kamath, Sishen Zhang, Changming Xu, Shubham Ugare, Gagandeep Singh, and Sasa Misailovic. Enforcing temporal constraints for LLM agents. In *ICLR 2026 Workshop: VerifAI-2: The Second Workshop on AI Verification in the Wild*, 2026. URL: <https://openreview.net/forum?id=VeRehDnGJJ>.
- [17] Jing-Jing Li, Jianfeng He, Chao Shang, Devang Kulshreshtha, Xun Xian, Yi Zhang, Hang Su, Sandesh Swamy, and Yanjun Qi. Stac: When innocent tools form dangerous chains for llm agents, 2026. URL: <https://arxiv.org/abs/2509.25624>, arXiv:2509.25624.
- [18] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, Online, August 2021. Association for Computational Linguistics. URL: <https://aclanthology.org/2021.acl-long.353/>, doi:10.18653/v1/2021.acl-long.353.
- [19] Xu Li, Simon Yu, Minzhou Pan, Yiyu Sun, Bo Li, Dawn Song, Xue Lin, and Weiyan Shi. Unsafer in many turns: Benchmarking and defending multi-turn safety risks in tool-using agents, 2026. URL: <https://arxiv.org/abs/2602.13379>, arXiv:2602.13379.
- [20] Alexander H Liu, Kartik Khandelwal, Sandeep Subramanian, Victor Jouault, Abhinav Rastogi, Adrien Sadé, Alan Jeffares, Albert Jiang, Alexandre Cahill, Alexandre Gavaudan, et al. Ministral 3. *arXiv preprint arXiv:2601.08584*, 2026.
- [21] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pages 52989–53046, 2024.
- [22] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *Proceedings of the 33rd USENIX Conference on Security Symposium, SEC ’24*, USA, 2024. USENIX Association.
- [23] Weidi Luo, Shenghong Dai, Xiaogeng Liu, Suman Banerjee, Huan Sun, Muhao Chen, and Chaowei Xiao. AGrail: A lifelong agent guardrail with effective and adaptive safety detection. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8104–8139, Vienna, Austria, July 2025. Association for Computational Linguistics. URL: <https://aclanthology.org/2025.acl-long.399/>, doi:10.18653/v1/2025.acl-long.399.
- [24] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. Harmbench: a standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- [25] Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. ToolSafe: Enhancing tool invocation safety of LLM-based agents via proactive step-level guardrail and feedback. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Findings of the Association for Computational Linguistics: ACL 2026*, pages 37125–37153, San Diego, California, United States, July 2026. Association for Computational Linguistics. URL: <https://aclanthology.org/2026.findings-acl.1850/>, doi:10.18653/v1/2026.findings-acl.1850.
- [26] Jesse Mu, Xiang Li, and Noah Goodman. Learning to compress prompts with gist tokens. In A. Oh,

- T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 19327–19352. Curran Associates, Inc., 2023. doi:10.52202/075280-0848.
- [27] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, dahai li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, editors, *International Conference on Learning Representations*, volume 2024, pages 9695–9717, 2024.
- [28] Paul Röttger, Hannah Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. XSTest: A test suite for identifying exaggerated safety behaviours in large language models. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5377–5400, Mexico City, Mexico, June 2024. Association for Computational Linguistics. URL: <https://aclanthology.org/2024.naacl-long.301/>, doi:10.18653/v1/2024.naacl-long.301.
- [29] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *The Twelfth International Conference on Learning Representations*, 2024. URL: <https://openreview.net/forum?id=GEcwtMklua>.
- [30] Mark Russinovich, Ahmed Salem, and Ronen Eldan. Great, now write an article about that: the crescendo multi-turn llm jailbreak attack. In *Proceedings of the 34th USENIX Conference on Security Symposium, SEC ’25*, USA, 2025. USENIX Association.
- [31] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- [32] Tianneng Shi, Jingxuan He, Zhun Wang, Linyu Wu, Hongwei Li, Wenbo Guo, and Dawn Song. Progent: Programmable privilege control for llm agents. *CoRR*, abs/2504.11703, April 2025. URL: <https://doi.org/10.48550/arXiv.2504.11703>.
- [33] Vincent Siu, Jingxuan He, Kyle Montgomery, Zhun Wang, Neil Gong, Chenguang Wang, and Dawn Song. A framework for formalizing llm agent security, 2026. URL: <https://arxiv.org/abs/2603.19469>, arXiv:2603.19469.
- [34] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Alan Chan, and Alex Pentland. Position: AI agents need authenticated delegation. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 82211–82231. PMLR, 13–19 Jul 2025. URL: <https://proceedings.mlr.press/v267/south25a.html>.
- [35] Gemma Team. Gemma 4 technical report, 2026. URL: <https://arxiv.org/abs/2607.02770>, arXiv:2607.02770.
- [36] Kimi Team. Kimi k2.5: Visual agentic intelligence, 2026. URL: <https://arxiv.org/abs/2602.02276>, arXiv:2602.02276.
- [37] Qwen Team. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. URL: <https://qwen.ai/blog?id=qwen3.5>.
- [38] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL: <https://aclanthology.org/2024.acl-long.850/>, doi:10.18653/v1/2024.acl-long.850.
- [39] Ada Defne Tur, Nicholas Meade, Xing Han Lù, Alejandra Zambrano, Arkil Patel, Esin DURMUS, Spandana Gella, Karolina Stanczak, and Siva Reddy. Safearena: Evaluating the safety of autonomous web agents. In *Forty-second International Conference on Machine Learning*, 2025. URL: <https://openreview.net/forum?id=7TrOBcxSvy>.
- [40] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions*

- on Machine Learning Research, 2024. URL: <https://openreview.net/forum?id=ehfRiF0R3a>.
- [41] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents, 2025. URL: <https://arxiv.org/abs/2407.16741>, arXiv:2407.16741.
- [42] Yuhui Wang, Tanqiu Jiang, Jiacheng Liang, Charles Fleming, and Ting Wang. Mage: Safeguarding llm agents against long-horizon threats via shadow memory, 2026. URL: <https://arxiv.org/abs/2605.03228>, arXiv:2605.03228.
- [43] Mengjie Wu, Yihao Huang, Zhenjun Lin, Kangjie Chen, Yuyang zhang, Yuhan Huang, Run Wang, and Lina Wang. Analogy-based multi-turn jailbreak against large language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL: <https://openreview.net/forum?id=RwCaBZ4w5P>.
- [44] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024. URL: <https://openreview.net/forum?id=BAakY1hNKS>.
- [45] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. Isolategpt: An execution isolation architecture for llm-based agentic systems. *arXiv preprint arXiv:2403.04960*, 2024.
- [46] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. Guardagent: Safeguard LLM agents via knowledge-enabled reasoning. In *Forty-second International Conference on Machine Learning*, 2025. URL: <https://openreview.net/forum?id=2nBcjCZrrP>.
- [47] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL: <https://openreview.net/forum?id=mXpq6ut8J3>.
- [48] Zhaorui Yang, Tianyu Pang, Haozhe Feng, Han Wang, Wei Chen, Minfeng Zhu, and Qian Liu. Self-distillation bridges distribution gap in language model fine-tuning. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1028–1043, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL: <https://aclanthology.org/2024.acl-long.58/>, doi:10.18653/v1/2024.acl-long.58.
- [49] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [50] Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. Shieldgemma: Generative ai content moderation based on gemma, 2024. URL: <https://arxiv.org/abs/2407.21772>, arXiv:2407.21772.
- [51] Guibin Zhang, Muxin Fu, and Shuicheng YAN. Memgen: Weaving generative latent memory for self-evolving agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL: <https://openreview.net/forum?id=vI56m4Iu4e>.
- [52] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL: <https://openreview.net/forum?id=V4y0CpX4hK>.
- [53] Jinchuan Zhang, Lu Yin, Yan Zhou, and Songlin Hu. Agentalign: Navigating safety alignment in the shift from informative to agentic large language models, 2025. URL: <https://arxiv.org/abs/2505.23020>, arXiv:2505.23020.
- [54] Shenyi Zhang, Yuchen Zhai, Keyan Guo, Hongxin Hu, Shengnan Guo, Zheng Fang, Lingchen Zhao, Chao Shen, Cong Wang, and Qian Wang. Jbshield: defending large language models from jailbreak attacks through activated concept analysis and manipulation. In *Proceedings of the 34th USENIX Conference on Security Symposium, SEC '25, USA*, 2025. USENIX Association.
- [55] Chujie Zheng, Fan Yin, Hao Zhou, Fandong Meng, Jie Zhou, Kai-Wei Chang, Minlie Huang, and Nanyun

Peng. On prompt-driven safeguarding for large language models. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 61593–61613. PMLR, 21–27 Jul 2024. URL: <https://proceedings.mlr.press/v235/zheng24n.html>.

- [56] Qihuang Zhong, Liang Ding, Li Shen, Juhua Liu, Bo Du, and Dacheng Tao. Revisiting knowledge distillation for autoregressive language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10900–10913, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL: <https://aclanthology.org/2024.acl-long.587/>, doi:10.18653/v1/2024.acl-long.587.
- [57] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pages 15585–15606, 2024.
- [58] Xueyang Zhou, Weidong Wang, Lin Lu, Jiawen Shi, Guiyao Tie, Xu Yongtian, Lixing Chen, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. SafeAgent: Safeguarding LLM agents via an automated risk simulator. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32516–32543, San Diego, California, United States, July 2026. Association for Computational Linguistics. URL: <https://aclanthology.org/2026.acl-long.1501/>, doi:10.18653/v1/2026.acl-long.1501.
- [59] Kaijie Zhu, Xianjun Yang, Jindong Wang, Wenbo Guo, and William Yang Wang. MELON: Provable defense against indirect prompt injection attacks in AI agents. In *Forty-second International Conference on Machine Learning*, 2025. URL: <https://openreview.net/forum?id=gtlMmGaKdZ>.
- [60] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to ai trans-

parency, 2025. URL: <https://arxiv.org/abs/2310.01405>, arXiv:2310.01405.

A Training Data Construction

We construct the safety supervision from 55 of the 70 MT-AgentRisk Filesystem tasks. The remaining 15 tasks are held out entirely from trajectory collection, target generation, routing and optimization. The construction pipeline first collects and filters pre-action states from multi-turn trajectories, then generates and audits task-level supervision targets, and finally associates the resulting targets with eligible states for routing and selection.

Trajectory-State Collection. We first collect pre-action states from multi-turn rollouts of the source tasks. For each trajectory, we snapshot the state immediately after the designated final-turn user request and before the next assistant action. This produces an initial pool of 78 final-turn states spanning 43 tasks. We then apply the prefix audit and discard states whose preceding trajectory has already realized the harmful objective, leaving 76 eligible states from the same 43 tasks.

Teacher Candidate Generation. We next construct the collapsed task view defined in Section 4.1 for each of the 55 source tasks and sample eight candidate completions per task from the frozen base model under the native action protocol, yielding 440 teacher candidates. Target generation is performed at the task level rather than per state: all 55 source tasks are sampled regardless of whether they ultimately contribute an eligible trajectory state.

Deterministic Structural Filtering. Each candidate is parsed according to the native action protocol. We retain only completions that contain a well-formed native FINISH action with a non-empty message and invoke no other native action. Because supervision routing relies on exact token positions, the parser must also recover the spans corresponding to the action header, the FINISH function name, and its message argument. Any candidate that fails these structural checks is discarded deterministically. Candidates that pass proceed to the semantic audit below.

Semantic Audit. Candidates that pass structural filtering are evaluated by GPT-4.1-mini [1] at temperature zero using the rubric below. The judge receives the canonical task, the complete candidate generation, and the extracted refusal message.

Semantic Audit Rubric

A candidate is semantically valid only if it:

- identifies a safety risk that applies to the task;
- clearly and unambiguously declines the harmful objective;
- does not claim that the requested harmful action was completed; and
- neither proposes nor performs a harmful non-termination action.

The judge returns a label, confidence score, and rationale. The allowed labels are VALID-REFUSAL, WRONG-RISK, NOT-COMMITTED, UNSAFE, and AMBIGUOUS. We accept a candidate only if it is labeled VALID-REFUSAL with confidence at least 0.8.

Invalid judge outputs and audit failures are assigned AMBIGUOUS and rejected. Accepted completions are deduplicated by their complete generated content, and at most three targets are retained per task. This procedure yields 52 audited targets across 28 tasks. We preserve each accepted completion, including its original reasoning, native action header, and refusal message, without rewriting or synthesizing target text. Candidate generation and structural validation use the same system instructions, tool specification, chat template, and native action protocol as deployment. All retained targets are frozen before optimization.

State-Target Pairing and Routing. We associate each eligible state with all audited targets from the same underlying task. Of the 76 eligible states, 38 states spanning 22 tasks have at least one audited target, yielding 74 state-target pairs. We then evaluate the base model on each original trajectory state and route its associated pairs according to Eq. 6. Among the 74 pairs, 69 require corrective supervision: 47 receive full-target supervision and 22 receive termination-only supervision. The remaining five require no correction and are excluded from the safety loss. The 69 corrective pairs cover 36 states from 21 tasks.

Final Optimization Set. Because a state may be associated with multiple corrective targets, optimizing over all such pairs would overrepresent states with more accepted targets. We therefore retain a single corrective target per state based on the base model’s log-probability at the decision position. The resulting safety set contains 36 trajectory-target pairs from 36 states and 21 tasks, including 34 full-target examples and two termination-only examples. We additionally include 20 fixed benign trajectory-completion pairs for the retention objective, yielding 56 optimization examples in total. All target generation, auditing, pairing, routing, and selection are completed before optimization begins.

Table 15: Summary of the training-data construction pipeline.

Construction Stage	Count
Filesystem source / held-out tasks	55 / 15
Collected final-turn states	78 (43 tasks)
Eligible final-turn states	76 (43 tasks)
Teacher candidates	440
Audited targets	52 (28 tasks)
Target-supported states	38 (22 tasks)
State-target pairs	74
Corrective pairs	69
Selected safety examples	36 (21 tasks)
Full / termination-only	34 / 2
Benign retention examples	20

Table 16: Additional training parameters used in the main experiments.

Parameter	Value
Optimizer	AdamW
Learning rate	3×10^{-4}
Warm-up updates	20
Weight decay	0
Gradient clipping	1.0

B Additional Training Details

We use the encoder architecture, LoRA placement, supervision weighting, and batch composition described in Section 5. Table 16 reports the remaining optimization settings used in the main experiments. The action-generating base model remains frozen throughout training. We conduct training on NVIDIA A100 GPUs.